

Overnight Prototype Plan

SyncVerse - what we build between now and 9 am, who builds it, and how we keep four people's work from colliding. The 33-page Master Build Blueprint stays the long-term target; this document replaces it for tonight.

VERDICT

The full blueprint is not feasible by 9 am. It is about 169 person-hours; tonight gives each person about 6 build hours, so roughly 24 in total. A prototype that shows **all seven problem-statement requirements at a basic level** is feasible: 18.5 hours of must-build work, 4.6 hours per person on average, 1.4 hours of slack each in a 6-hour window. Estimates are rough, plus or minus 40%, so the plan is built to shrink safely (section 2.3).

What the judges see at 9 am (about 5 minutes)

1. Three people join one room by code; everyone edits one Python file with live labelled cursors; video and chat run in the same tab.
2. Each person runs the same file with different input and sees only their own output.
3. A student's run fails; the error line is marked; Explain with AI gives a plain-language card; Show patch opens a diff; Accept updates everyone's editor, Reject changes nothing.
4. The quality panel lists formatting, naming, smell, complexity and security findings.
5. A mentor requests debug access; the student allows; the mentor sees the student's console; the student revokes and the view disappears.
6. The progress page shows runs, error categories and observation sentences (no scores).
7. We close with the roadmap: execution canvas, predict-and-reflect checkpoints, classrooms. We say plainly these are not built yet.

Coverage of the seven requirements

Requirement	What exists at 9 am	What is honestly missing
Real-time editing	Yjs sync, labelled cursors, presence, one shared file	Multiple files, upload, version history
Video chat	LiveKit: video, audio, screen share, chat	Custom layout, low-bandwidth mode
AI error help	Explain card, line highlight, patch diff with Accept / Reject	Hint mode unless ahead (SHOULD)
Sandboxed run	Judge0 Python with stdin, limits, private console per person	Other languages unless ahead (SHOULD)
Code quality	Six Python rules in a grouped panel, markers in the editor	Other languages, maintainability summary
Collaborative debugging	Request, allow, revoke; mentor sees console and error	Step debugger, execution canvas, shared playback
Progress tracking	Events, student page, mentor table, observation sentences	Class trends, adaptive recommendations

1. Clock, checkpoints and capacity

Clock times assume kickoff at **5:30 pm**. If you start later, shift the plan and apply the cut ladder in section 2.3; do not compress the sleep window below four hours, because bugs written at 3 am cost more than the features they add.

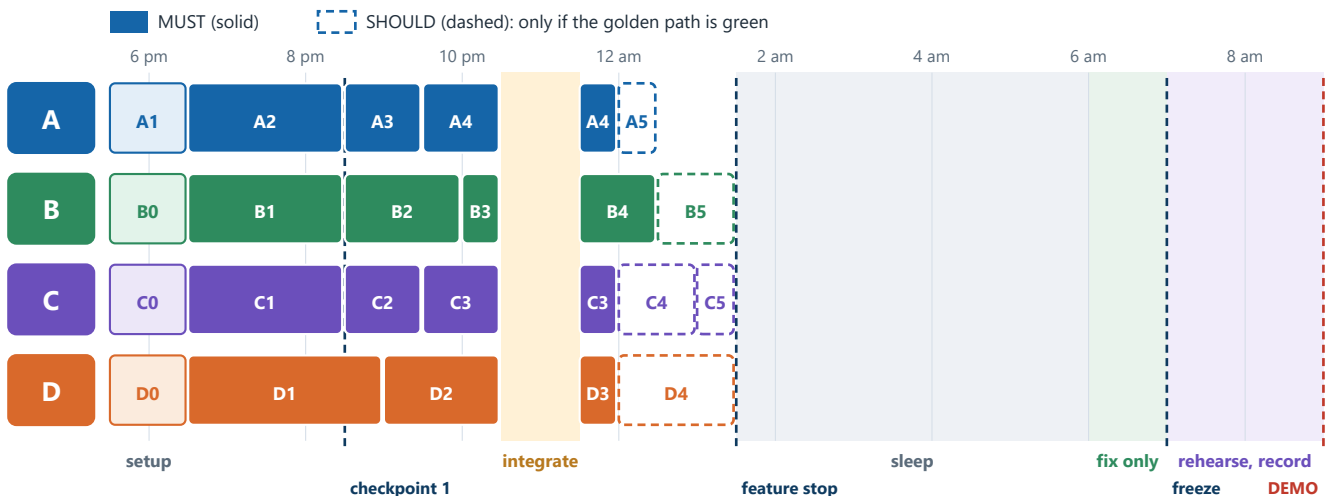


Figure 1 - Every lane at a glance. Bars are labelled with task IDs (A2 = P-A2). The amber column is the only time everyone stops building to integrate.

When	What happens	Exit test
5:30 - 6:30 pm	Phase 0 (everyone). A builds the skeleton and pushes it to main. B, C, D get their external service working from a scratch script (P-B0, P-C0, P-D0). Nobody writes app code before the skeleton is on main.	Skeleton runs on all four laptops; each service answers one real call
6:30 - 10:30 pm	Build 1. Each lane works only in its own folders against stubs and contracts (section 4).	-
8:30 pm	Checkpoint 1. First real slice of every lane merged to main: A's synced editor, B's run API, C's explain API (D's debug API follows at 9 pm).	Main starts and the smoke test passes
10:30 - 11:30 pm	Integration window (all hands). Run the full golden path together. Log every break against a lane. Fix contract mismatches first.	Steps 1 to 5 of the demo work end to end
11:30 pm - 1:30 am	Build 2. Finish every MUST. Start a SHOULD only if the golden path is green on main.	All MUST merged
1:30 am	Feature stop. Merge, tag night-1, then sleep. Only bug fixes after this point.	Tag exists and runs
1:30 - 6:00 am	Sleep (at least four hours).	-
6:00 - 7:00 am	Fix only. Run the smoke test and the golden path; fix what broke overnight.	Golden path passes twice
7:00 am	Freeze. Tag demo-1. Nothing merges to main after this except a fix the lead approves.	-
7:00 - 8:00 am	Rehearse twice, record the backup video, load seed data, warm up every service.	Video plays offline
8:00 - 9:00 am	Buffer and setup at the venue. No code changes.	-

Capacity math (computed from the task list)

Lane	Must-build hours	Should hours	Window	Slack	Must-build ends
A Editor sync, presence, video	4.5	0.5	6.0	1.5	12 am
B Run pipeline, console, quality	5.0	1.0	6.0	1.0	12:30 am
C AI explain and patch	4.5	1.5	6.0	1.5	12 am
D Debug access, progress, demo data	4.5	1.5	6.0	1.5	12 am
Total	18.5	4.5	24.0	5.5	

Window = 18.5 to 22.5 plus 23.5 to 25.5 = 6.0 build hours per person. Phase 0 (1 hour each) and the integration hour are outside it. Lane B has the least slack; if B falls behind, C or D takes B5 or B4 first.

2. Scope for tonight

MUST is what the golden path needs. **SHOULD** is taken in order only when the golden path is green on main. Everything else is cut. Blueprint task IDs are shown so nothing is lost from the long-term plan.

2.1 MUST and SHOULD

ID	Lane	Task	Tier	Hrs	Blueprint
A2	A	Synced editor and rooms	MUST	2.0	T-A-02, T-A-03, T-A-04 (lite)
A3	A	Cursors and presence	MUST	1.0	T-A-05
A4	A	Video dock	MUST	1.5	T-A-07, T-D-12
A5	A	Persist documents, reconnect banner	SHOULD	0.5	T-A-03, T-A-12
B1	B	Run pipeline	MUST	2.0	T-B-01
B2	B	Console panel	MUST	1.5	T-B-02, T-B-03 (lite)
B3	B	Error line marking	MUST	0.5	T-B-04 (lite)
B4	B	Quality analysis (Python, six rules)	MUST	1.0	T-C-04, T-C-05 (lite)
B5	B	More languages	SHOULD	1.0	T-B-05
C1	C	LLM gateway and explain	MUST	2.0	T-C-01, T-C-02
C2	C	AI panel	MUST	1.0	T-C-02
C3	C	Patch preview, Accept or Reject	MUST	1.5	T-C-03
C4	C	Hint mode	SHOULD	1.0	T-C-06
C5	C	AI answer check	SHOULD	0.5	T-C-10
D1	D	Permission-gated debug access	MUST	2.5	T-D-02 (lite)
D2	D	Learning events and progress	MUST	1.5	T-D-03, T-D-04 (lite)
D3	D	Samples and seed data	MUST	0.5	T-D-07 (lite)
D4	D	Mentor overview tiles	SHOULD	1.5	T-A-09, T-D-09

2.2 Cut entirely tonight

- **Accounts, database, row-level security, deployment.** State lives in server memory and the server runs on one laptop. Privacy is enforced in the API (owner or active grantee only), not in a database.
- **Execution canvas, Python tracer, shared playback, predict-and-reflect** (T-B-06 to T-B-08, T-D-06; about 13 hours). They are the signature ideas, so show them as a roadmap slide, not as a half-built demo.
- **Multiple files, uploads, version history, class trends, AI code review and explain-code, micro-lessons, landing page, accessibility pass, low-bandwidth mode** and everything in blueprint sections 4 and 5.

2.3 Cut ladder: if you lose hours

Each hour lost from the evening removes about four person-hours. Apply in this order and stop as soon as the plan fits. The last row is what we never cut.

Step	Cut	Frees	What the demo does instead
1	All SHOULD tasks	4.5 h	Nothing visible changes
2	P-D3 samples menu (paste samples by hand); seed a fixed progress state	0.5 h	Presenter pastes the planted-bug program

Step	Cut	Frees	What the demo does instead
3	P-A4 video dock	1.5 h	A normal video call in another window, said out loud
4	P-B4 lint down to three rules	0.5 h	Fewer findings listed
5	P-D2 observations: counts only, no sentences	0.5 h	Bars and counts, no sentences
6	P-C3 patch: diff preview only, no Accept	0.5 h	Show the diff, apply by hand
Never	P-A2 + P-A3 (editor), P-B1 to P-B3 (run and error line), P-C1 + P-C2 (explain), P-D1 (debug access)	-	These four ideas are the demo

3. Prototype architecture

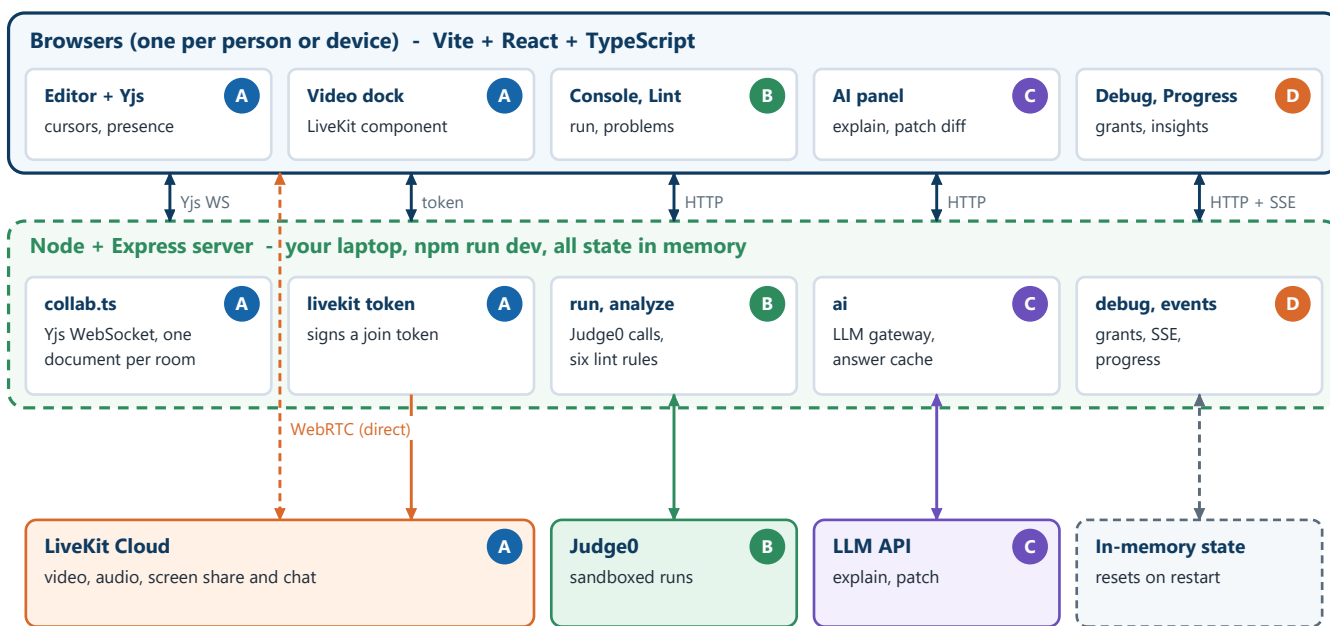


Figure 2 - Everything runs on one laptop with `npm run dev`; only Judge0, the LLM and LiveKit are external. Circles show the lane that owns each part.

WHY THIS DIFFERS FROM THE BLUEPRINT

Tonight's stack (Vite + Express + in-memory state) is deliberately simpler than the blueprint's (Next.js, Supabase, Render, Vercel). No hosted database, no deployments, and only three accounts to set up. Nothing tonight blocks moving to the blueprint stack later: contracts in `shared/types.ts` carry over.

3.1 Stack and packages

Part	Choice	Note
Web	Vite, React, TypeScript, Tailwind	Fast dev server; proxy <code>/api</code> and <code>/collab</code> to the server
Editor	monaco-editor, @monaco-editor/react, yjs, y-monaco, y-websocket	Check the y-websocket README: the standalone server helper may live in a separate package in recent versions
Server	Node LTS, Express, ws, zod, tsx	One process: REST, SSE, Yjs WebSocket
Video	livekit-server-sdk (server), @livekit/components-react + livekit-client (web)	Prebuilt conference UI gives chat and screen share
AI	Any provider with JSON output behind one adapter	With Claude: claude-haiku-4-5-20251001 for explain, claude-sonnet-5-5 for patch
Run	Judge0 REST	Read language ids from <code>/languages</code> ; use base64 and polling

Environment variables (names only; values go in `.env`, which is git-ignored): `PORT`, `JUDGE0_URL`, `JUDGE0_API_KEY`, `JUDGE0_API_HOST` (if the provider needs it), `LLM_API_KEY`, `LLM_MODEL_FAST`, `LLM_MODEL_STRONG`, `LIVEKIT_URL`, `LIVEKIT_API_KEY`, `LIVEKIT_API_SECRET`.

3.2 Repository layout and who edits what

```

syncverse/
  shared/types.ts      contracts (A owns; changes announced in the group chat first)
  web/src/App.tsx      shell with one slot per panel (A, edited once in P-A1)
  web/src/session.ts   session + WorkspaceContext + usePresence (A)
  web/src/editor/      video/      Lane A
  web/src/console/     quality/    Lane B
  web/src/ai/          Lane C
  web/src/debug/       progress/   demo/      Lane D
  server/index.ts      mounts every router (A, edited once in P-A1)
  server/collab.ts     Yjs WebSocket (A)      server/routes/livekit.ts (A)
  server/routes/run.ts server/routes/analyze.ts (B)
  server/routes/ai.ts  (C)
  server/routes/debug.ts server/routes/events.ts (D)
  docs/TRACKER.md     one line per task; every lane edits only its own rows

```

The rule that prevents most merge trouble: App.tsx and server/index.ts import each lane's folder index and never change again after P-A1, so nobody edits the same file twice. Cross-lane calls go through exported functions with fixed signatures (section 4.3), which the skeleton provides as stubs.

4. Contracts (fixed before anyone builds)

4.1 Session and headers

No login tonight. On the join screen each person enters a name, picks student or mentor and types a room code; the web app stores {userId (uuid), name, color, role, roomCode} in localStorage and sends x-user-id, x-user-name and x-role on every request. The server trusts these. That is acceptable for a prototype and must be said if asked.

4.2 Shared types (shared/types.ts)

```

export type RunStatus = 'queued'|'running'|'success'|'compile_error'|'runtime_error'
                        |'timeout'|'memory_limit'|'service_error';
export interface RunResult { id:string; ownerId:string; roomCode:string; language:string;
  source:string; stdin:string; status:RunStatus; stdout:string; stderr:string;
  compileOutput:string; timeMs?:number; memoryKb?:number;
  errorLine?:number; errorMessage?:string; createdAt:number }
export interface Diagnostic { category:'formatting'|'naming'|'smell'|'complexity'|'security';
  severity:'info'|'warning'|'error'; line:number; col?:number; rule:string; message:string }
export interface Explanation { what:string; whereLine?:number; why:string; plain:string;
  fix:string; snippet?:string; concepts:string[] }
export interface DebugGrant { id:string; ownerId:string; granteeId:string; granteeName:string;
  status:'requested'|'active'|'denied'|'revoked'|'expired'; createdAt:number; expiresAt?:number }
export interface LearningEvent { userId:string; roomCode:string; at:number;
  type:'run'|'explain'|'patch'|'lint'|'debug_access'; category?:string; concepts?:string[]; ok?:boolean }
export interface EditorHandle { getValue():string; replaceAll(text:string):void;
  setMarkers(m:{line:number;message:string;severity:'error'|'warning'|'info'}[]):void;
  highlightLine(line:number|null):void }

```

React side: useEditor() returns the EditorHandle, useWorkspace() returns {lastRun, setLastRun}, usePresence() returns [{userId, name, color, role, state}]. All three exist as stubs in P-A1.

4.3 Endpoints and cross-lane functions

Route	Body or query	Returns	Lane
WS /collab/:roomCode	Yjs protocol	synced document	A
GET /api/livekit/token	room, name, identity	{token, url}	A
POST /api/run	{roomCode, language, source, stdin}	{id}	B
GET /api/run/:id	-	RunResult (owner or active grantee, else 403)	B
GET /api/runs/latest	ownerId	RunResult or null (owner or grantee)	B

Route	Body or query	Returns	Lane
POST /api/analyze	{source}	Diagnostic[]	B
POST /api/ai/explain	{runId}	Explanation	C
POST /api/ai/patch	{runId}	{patchedSource}	C
POST /api/debug/request	{ownerId}	DebugGrant	D
POST /api/debug/:id/decision	{allow:boolean}	DebugGrant	D
POST /api/debug/:id/revoke	-	DebugGrant	D
GET /api/debug/events	SSE, per user	grant changes	D
GET /api/progress/me, /api/progress/room	-	aggregates and observations	D

```

// server/routes/events.ts (D) export function logEvent(e: LearningEvent): void
// server/routes/debug.ts (D) export function canView(viewerId: string, ownerId: string): boolean
// server/routes/run.ts (B) export function getRun(id: string): RunResult | undefined
// export function getLatestRunFor(ownerId: string): RunResult | undefined

```

4.4 Judge0 details that save time

Judge0 status id	Our RunStatus	Note
1, 2	queued, running	keep polling every 500 ms, give up at 15 s as service_error
3	success	we send no expected output, so 3 means finished cleanly
5	timeout	CPU limit 5 s
6	compile_error	compile_output holds the message
7 to 12	runtime_error, or memory_limit if peak memory is at the 128 MB limit	Judge0 has no separate memory-limit status
13, 14	service_error	offer retry
any	compile_error if stderr contains SyntaxError or IndentationError	Python reports syntax errors at run time, so Judge0 calls them runtime errors

Cap stdout at 64 KB, stdin at 10 KB. Rewrite nothing for Python. Progress observation rule: three or more failures of the same error category in the last ten runs and the latest run not successful produces 'Retry recommended: loop boundaries' (IndexError maps to lists-arrays and loop-boundaries, NameError to variables, SyntaxError to syntax-basics, RecursionError to recursion, timeout to loops).

5. Lane cards

Each card is the whole job for one person tonight. Fill in owner and backup now. **Start without waiting:** B, C and D build against the stubs from P-A1 and the contracts above, using fixtures or curl, and connect to the real editor at checkpoint 1.

Lane A - Editor sync, presence, video Owner: _____ Backup: _____				
ID	Window	Task and what to build	Hrs	Needs
A1 SETUP	5:30p-6:30p	Skeleton on main npm workspaces (web, server, shared). Vite + React + TypeScript with plain Monaco behind the EditorHandle. Express server with empty routers mounted for run, analyze, ai, debug, events, livekit. Shell layout with one named slot per panel. shared/types.ts, session context, WorkspaceContext, usePresence stub, stubs of logEvent and canView. Install every dependency in section 3.1 now so nobody edits package.json later. Vite proxy for /api and /collab. .env.example, .gitignore, docs/TRACKER.md. Done when: A fresh clone runs npm install && npm run dev and shows the shell. B, C and D have pulled it and can run it.	1.0	-

ID	Window	Task and what to build	Hrs	Needs
A2 MUST	6:30p-8:30p	Synced editor and rooms server/collab.ts: Yjs WebSocket, one document per room code. Web: join screen (name, role student or mentor, room code), Monaco bound to the room's Y.Text with y-monaco, EditorHandle implemented on top of it. replaceAll is one Y transaction. Single shared file, Python mode. Done when: Two browsers in the same room see each other's typing with no lost characters; setMarkers and highlightLine work from the console.	2.0	P-A1
A3 MUST	8:30p-9:30p	Cursors and presence Labelled coloured remote cursors and selections from Yjs awareness (y-monaco needs a little CSS per user). Presence list with online / typing / idle, exposed through usePresence() for Lane D. Done when: A collaborator's name label follows their cursor and the list shows three people.	1.0	P-A2
A4 MUST	9:30p-10:30p, 11:30p-12a	Video dock server/routes/livekit.ts mints a token. Web: LiveKit prebuilt conference component in a collapsible dock. It already includes camera and mic toggles, screen share and in-room chat. Done when: Two devices see and hear each other, screen share works, chat works.	1.5	P-D0 keys
A5 SHOULD	12a-12:30a	Persist documents, reconnect banner Save Yjs state to a JSON file on change (debounced) and reload it at start. Show a banner when the WebSocket drops. Done when: Restarting the server keeps the code; unplugging Wi-Fi shows the banner.	0.5	P-A2

Lane B - Run pipeline, console, quality Owner: _____ Backup: _____				
ID	Window	Task and what to build	Hrs	Needs
B0 SETUP	5:30p-6:30p	Judge0 hello-world Get Judge0 access (hosted key or any endpoint you can reach). From a scratch script submit print(1) and a stdin program, poll to completion, note the Python language id from /languages, check the daily quota. If hosted Judge0 is not working by 7:30 pm, switch to the local-subprocess fallback (section 7.3) and tell everyone. Done when: A scratch script prints the stdout of a Python program run through Judge0, and the quota is written in the tracker.	1.0	-
B1 MUST	6:30p-8:30p	Run pipeline server/routes/run.ts: POST /api/run submits to Judge0 (base64, 5 s CPU, 128 MB, stdin), the server polls until done, stores the result, maps statuses (section 4.4). Exports getRun and getLatestRunFor; reads are allowed only for the owner or an active grantee via canView. Calls logEvent. Done when: Hello-world and a stdin program return correct output; an infinite loop returns timeout; a SyntaxError returns compile_error with stderr.	2.0	P-B0
B2 MUST	8:30p-10p	Console panel Run button (Ctrl+Enter) sending the editor text, stdin box, stdout / stderr tabs, status badge with time and memory, last five runs, a 'only you can see this' lock. Sets lastRun in WorkspaceContext so Lane C can read it. Done when: Three people run the same file with different stdin and each sees only their own output.	1.5	P-B1; editor text via EditorHandle (stub ok)
B3 MUST	10p-10:30p	Error line marking Parse the Python traceback into errorLine and errorMessage; call setMarkers and highlightLine; clicking the error jumps to the line. Done when: The off-by-one IndexError sample marks the correct line.	0.5	P-B2, P-A2 handle
B4 MUST	11:30p-12:30a	Quality analysis (Python, six rules) server/routes/analyze.ts, regex and indentation heuristics on the text: line over 100 chars, one-letter names, nesting deeper than 3, magic numbers, eval / exec / os.system, bare except. Panel groups findings by category, click jumps to the line, markers appear in the editor. Runs 1.5 s after typing stops. Done when: The quality sample triggers formatting, naming, smell, complexity and security findings.	1.0	P-A2 handle

ID	Window	Task and what to build	Hrs	Needs
B5 SHOULD	12:30a-1:30a	More languages Language selector for C, C++, Java, JavaScript with the Judge0 ids from /languages, compiler flags, and the Java class-name rewrite to Main. Done when: Each language prints hello-world and reads stdin.	1.0	P-B1

Lane C - AI explain and patch Owner: _____ Backup: _____

ID	Window	Task and what to build	Hrs	Needs
C0 SETUP	5:30p-6:30p	LLM hello-world Get an LLM key and set a spending cap. From a scratch script make one call that returns JSON matching the Explanation type and time it. Draft the prompt against the IndexError sample. Done when: One structured Explanation comes back in under 6 s and the key is in .env, not in git.	1.0	-
C1 MUST	6:30p-8:30p	LLM gateway and explain server/routes/ai.ts: provider adapter, code and error sent as quoted data, zod-validated JSON with one retry, 15 s timeout, cache by hash of language + error + code, and pre-seeded answers for the planted samples. POST /api/ai/explain {runId} (owner or grantee only) returns an Explanation. Calls logEvent. Done when: The NameError, IndexError and SyntaxError samples return a correct beginner-level explanation pointing at the right line, cached ones instantly.	2.0	P-C0
C2 MUST	8:30p-9:30p	AI panel Explain with AI button on any failed run (reads lastRun), card with what / where / why / fix / snippet, line highlight via the editor, loading and error states, 'AI can be wrong' note. Done when: A failed run becomes a card and a highlighted line in one click.	1.0	P-C1, P-B2 (stub lastRun ok)
C3 MUST	9:30p-10:30p, 11:30p-12a	Patch preview, Accept or Reject POST /api/ai/patch {runId} returns the corrected source (stronger model, minimal change, rejected if empty or unrelated). Web: Monaco DiffEditor in a modal; Accept calls replaceAll so every collaborator gets it; Reject changes nothing; if the document changed since the run, regenerate. Logs the decision. Done when: Reject leaves the document untouched; Accept appears in the second browser.	1.5	P-C1, P-A2 handle
C4 SHOULD	12a-1a	Hint mode Three-tier ladder in the AI panel: nudge, guiding question, near-answer. Tiers 1 and 2 must not contain code (prompt rule plus a check for code fences). Done when: Tier 1 and 2 never show the fix.	1.0	P-C2
C5 SHOULD	1a-1:30a	AI answer check Run the eight planted samples, read the answers, fix prompts, refresh the cache. Done when: At least 7 of 8 answers are correct and fast.	0.5	P-C1

Lane D - Debug access, progress, demo data Owner: _____ Backup: _____

ID	Window	Task and what to build	Hrs	Needs
D0 SETUP	5:30p-6:30p	LiveKit keys and samples Create a LiveKit Cloud project, mint one token from a scratch script, and hand URL + keys to Lane A for P-A4. Write the planted-bug sample programs (section 7.2). Sketch the debug-access states on paper. Done when: A scratch script prints a valid token; keys are shared with A; five sample .py files exist.	1.0	-
D1 MUST	6:30p-9p	Permission-gated debug access server/routes/debug.ts: in-memory grants (request, allow, deny, revoke, 30 min expiry), canView(viewerId, ownerId), and an SSE stream per user. Web: a Request access button per person (from usePresence), a modal for the owner (Allow / Deny), a banner 'X is viewing your session - Revoke', and a read-only view of the owner's latest run for the grantee. Done when: Request, allow, mentor sees the student's output, revoke clears it instantly, and a third user gets 403 on the same data.	2.5	P-A1 stubs; real runs from P-B1

ID	Window	Task and what to build	Hrs	Needs
D2 MUST	9p-10:30p	Learning events and progress server/routes/events.ts owns logEvent and aggregates; progress page shows runs, success rate, error categories (simple bars), AI help used, and observation sentences from the rule in section 4.4. Mentor table: per student runs, failures, top error. No scores or ranks; a 'what we record' note. Done when: After the demo flow the page shows believable observations.	1.5	P-A1 stubs; events from B and C
D3 MUST	11:30p-12a	Samples and seed data Planted-bug sample menu that loads a program via replaceAll, and seed events so the progress page has history on a fresh start. Done when: Picking a sample loads it for everyone; a fresh start shows seeded history.	0.5	P-A2 handle, P-D2
D4 SHOULD	12a-1:30a	Mentor overview tiles One tile per participant: presence, last-run status, failed-run streak, stuck flag after three failures, Request access button. Status only, never output. Done when: A mentor spots a stuck student without opening their session.	1.5	P-D1, P-D2

NOTE

Lane D is where lanes meet: it reads presence from A, runs from B and events from C. D should get real data from B and C at checkpoint 1; until then use the stubs and seeded runs.

6. Working together: branches, merging, smoke test

6.1 Are the lanes separate? Will merging be error-free?

STRAIGHT ANSWER

Mostly separate in code, dependent in behaviour, and not automatically error-free. Git only detects text conflicts. It will happily merge B renaming a field that C still reads, and the app will break at runtime. The plan removes most risk with six rules; the smoke test catches the rest.

Risk	What prevents it
Two people edit the same file	Folder ownership (3.2). App.tsx and server/index.ts never change after P-A1.
package.json / lockfile conflicts	Every dependency is installed in P-A1. If you must add one, tell the chat, add it in a tiny PR, everyone merges main at once. If the lockfile conflicts: take main's copy, run npm install, commit.
Contract drift (renamed fields, changed shapes)	shared/types.ts is owned by A. Changes are announced in the chat before the PR; additive changes only.
Line-ending noise on Windows	Everyone runs <code>git config core.autocrlf input</code> once.
Late surprises	Checkpoint 1 at 8:30 pm and the integration window at 10:30 pm force real connections early.
Main breaks	Merge only after the smoke test passes on your branch merged with latest main. If main is red, the author fixes or reverts within 10 minutes.

6.2 Git routine

```
git checkout main && git pull
git checkout -b lane-b/P-B1-run-pipeline      # one branch per task
... work, commit small ...
git fetch origin && git merge origin/main      # every hour, and before every push
git push -u origin lane-b/P-B1-run-pipeline  # open a PR; self-merge is allowed if smoke test passes
git checkout main && git pull                  # everybody pulls main after any merge
```

- Never push directly to main and never force-push. Do not commit .env.
- Commit messages start with the task ID, for example P-B1: poll Judge0 until done.
- Update your row in docs/TRACKER.md in the same PR (TODO, DOING, MERGED, BLOCKED with a one-line note).

6.3 Smoke test (2 minutes, run after every merge to main)

1. npm install && npm run dev starts web and server with no errors in either console.
2. Open two tabs, join the same room code with different names.
3. Type in one tab; the text appears in the other, and a name label follows the cursor.
4. Run print('hi') in both tabs with different stdin; each sees only its own output.
5. Run the IndexError sample; the error line is marked.
6. Click Explain with AI; a card appears. Click Show patch, Reject, then again Accept; the other tab updates.
7. Mentor requests debug access; student allows; mentor sees the console; student revokes.
8. Open the progress page; it shows the runs just made.

Steps 5 to 8 apply as those lanes land. If a step fails on main, the lane that owns it fixes it before anyone merges more.

7. Demo, fallbacks and honesty

7.1 Golden path script

#	Who	What happens	Tasks
1	Mentor	Creates room Loops-101; two students join with the code	A2
2	All	Edit together; cursors visible; video and chat on screen	A2 A3 A4
3	Students	Run the same file with different stdin; separate consoles	B1 B2
4	Student 1	Loads the off-by-one sample, runs it, line is marked, Explain, Show patch, Reject, then Accept	B3 C1 C2 C3 D3
5	Student 2	Quality panel shows findings; fix one and it disappears	B4
6	Mentor, Student 1	Request access, Allow, console visible, Revoke	D1
7	Mentor	Progress page and observation sentence	D2
8	Lead	Roadmap slide: execution canvas, predict-and-reflect, classrooms. Not built yet	-

7.2 Planted-bug samples (P-D0 writes them, P-D3 loads them)

- IndexError: loop runs to len(nums) + 1 (the golden-path sample).
- NameError: a misspelled variable. SyntaxError: a missing colon.
- Infinite loop (timeout). RecursionError: no base case.
- Quality sample: nested loops, one-letter names, magic number, long line, eval.

7.3 Fallbacks

If this breaks	Do this	Decide by
Hosted Judge0 unavailable or over quota	Run Python with a local subprocess (timeout 5 s) on the server. Label it 'demo runner, not sandboxed'; never claim sandboxing for it.	7:30 pm (B)
LLM slow or down	Pre-seeded cache answers for the samples make the demo work offline; say so if asked.	P-C1
Yjs or cursor CSS fights you	Drop label styling before dropping sync: plain colour cursors are enough.	P-A3
LiveKit not working	Use a normal video call in another window and say so.	9:30 pm (A)
Venue Wi-Fi	Phone hotspot, server on your laptop, other devices join by its address. Use a second device for the second camera so two tabs do not fight over one webcam.	7:30 am
A lane is behind at 10:30 pm	Apply the cut ladder (2.3) immediately; do not wait for morning.	Lead

7.4 Say this if asked (prototype shortcuts)

- No login yet; identity is a name and a room code. State lives in server memory and resets on restart unless P-A5 is done.
- Privacy is enforced in the API, not in a database. Only Python runs unless P-B5 is done. Quality rules and progress observations are simple heuristics.
- AI explanations can be wrong, and the student always decides whether a patch is applied.
- It runs locally, not deployed. The roadmap covers the canvas, predict-and-reflect, classrooms, a real database and accounts.

8. Setup checklist and risks

8.1 Before 6:30 pm

- Everyone: Node LTS installed, git installed, `git config core.autocrlf input`, GitHub account, editor open.
- A: create the private GitHub repo, add the other three as collaborators, push the skeleton (P-A1).
- B: Judge0 access working (P-B0). C: LLM key with a spending cap (P-C0). D: LiveKit project and keys, handed to A (P-D0).
- Keys go in a shared password manager or direct message, never in git, never in the group chat history that others can forward.

8.2 Top risks

Risk	L	I	Mitigation
Judge0 access or quota	M	H	P-B0 tests it first; local-subprocess fallback decided by 7:30 pm
Integration breaks at 10:30 pm	H	M	Contracts, stubs, checkpoint 1, smoke test, integration window with a fix-first rule
Estimates too low (plus or minus 40%)	H	H	6 h window vs 4.6 h per person; cut ladder; SHOULDs are dropped first
Yjs, Monaco, y-monaco quirks	M	M	A spikes cursors first; degrade labels before sync
LLM latency, cost or wrong answers	M	M	Cache, fast model, spending cap, pre-seeded samples, note that AI can be wrong
Sleep debt causes bad merges	M	H	Hard feature stop at 1:30 am, at least four hours of sleep, freeze at 7 am
Secrets leaked to git	L	H	.env git-ignored from P-A1; review the first push

L = likelihood, I = impact (H high, M medium, L low).

9. Tracker (copy into docs/TRACKER.md)

Mark status as TODO, DOING, MERGED or BLOCKED. Tick the box when the task's 'Done when' passes on main.

	ID	Lane	Task	Tier	Owner	Status
☐	A1	A	Skeleton on main	SETUP		TODO
☐	B0	B	Judge0 hello-world	SETUP		TODO
☐	C0	C	LLM hello-world	SETUP		TODO
☐	D0	D	LiveKit keys and samples	SETUP		TODO
☐	A2	A	Synced editor and rooms	MUST		TODO
☐	A3	A	Cursors and presence	MUST		TODO
☐	A4	A	Video dock	MUST		TODO
☐	A5	A	Persist documents, reconnect banner	SHOULD		TODO
☐	B1	B	Run pipeline	MUST		TODO
☐	B2	B	Console panel	MUST		TODO
☐	B3	B	Error line marking	MUST		TODO
☐	B4	B	Quality analysis (Python, six rules)	MUST		TODO
☐	B5	B	More languages	SHOULD		TODO
☐	C1	C	LLM gateway and explain	MUST		TODO
☐	C2	C	AI panel	MUST		TODO
☐	C3	C	Patch preview, Accept or Reject	MUST		TODO
☐	C4	C	Hint mode	SHOULD		TODO
☐	C5	C	AI answer check	SHOULD		TODO
☐	D1	D	Permission-gated debug access	MUST		TODO
☐	D2	D	Learning events and progress	MUST		TODO
☐	D3	D	Samples and seed data	MUST		TODO
☐	D4	D	Mentor overview tiles	SHOULD		TODO